TrendMart

Sales & Revenue Analytics

A Complete Story: Raw CSV → Python ETL → MySQL Star Schema → Power BI

Prepared by:

Harshita Fogat

harsfogat.ext@deloitte.com

1. Project Objectives & Business Context

1.1 Problem Statement

Imagine TrendMart, a small retail business struggling to boost sales and maximise revenue. Despite having a diverse product range and a loyal customer base, the business owner faces three recurring challenges:

- **Fluctuating sales trends:** some products perform well while others struggle, with no visibility into why.
- **Customer spending patterns:** some customers buy frequently while others rarely return, and no segmentation exists to act on this.
- **Revenue optimization issues:** discounts and promotions are in place but their true impact on profitability is unknown.

This project builds an end-to-end analytical pipeline: two raw CSV files are extracted, cleaned, and feature-engineered in Python; stored in a nine-table MySQL star schema; and delivered as five interactive Power BI dashboard pages.

1.2 Business Goals

1. **Analyse sales trends** across products, categories, cities, and time to identify growth opportunities.
2. **Identify customer behaviour** by segmenting customers using RFM methodology and K-Means clustering.
3. **Forecast future revenue** using SARIMA time-series modelling to support inventory planning.
4. **Measure promotion impact** to determine whether the discount policy improves or erodes profitability.

1.3 Technology Stack

Each technology plays a distinct role in the pipeline:

- **Python (pandas, numpy, scikit-learn, statsmodels):** Data extraction, transformation, feature engineering, RFM + K-Means clustering, SARIMA forecasting, cohort analysis, and EDA. All implemented in a single Jupyter Notebook structured as a linear ETL pipeline with structured logging at every stage.
- **MySQL (SQLAlchemy + PyMySQL):** Star schema data warehouse (trendmart_dw). Nine tables loaded via to_sql() with if_exists='replace' for idempotency. All date columns stored as MySQL DATE type for automatic Power BI import.
- **Power BI (Power Query + DAX):** Five dashboard pages. Data model mirrors the star schema with five active relationships. Fifteen DAX measures in an isolated _Measures table. Four calculated columns added in Power Query.
- **scikit-learn (StandardScaler + KMeans):** RFM features standardised before clustering to prevent scale dominance. KMeans fit on k=3 selected via elbow method.
- **statsmodels (SARIMAX):** SARIMA(1,1,1) model for 3-month forward forecasting. Produces 95% confidence interval bounds stored as CI_Lower and CI_Upper.
- **SQL (analytical validation):** Six queries executed directly against MySQL post-load to cross-validate all Python EDA findings.

1.4 Key Performance Indicators (KPIs)

The following KPIs were identified from the case study requirements and supplemented with additional indicators derived during analysis:

KPI #	KPI Name	Definition & Business Value
1	Total Net Revenue	Sum of all Net_Amount values after discount. Headline revenue figure: $1,670,762.22
2	Total Gross Revenue	Sum of Unit_Price × Quantity before discounts. Baseline before promotions: $1,782,148.00
3	Total Transactions	Count of all Transaction_IDs. Volume indicator: 2,705
4	Total Units Sold	Sum of Quantity across all transactions. Demand indicator: ~6,900 units
5	Unique Customers	DISTINCTCOUNT of Customer_ID. Base size: 1,000
6	Average Order Value (AOV)	Total Revenue ÷ Total Transactions. Basket size: $617.66
7	Revenue per Customer	Total Revenue ÷ Unique Customers. Customer value: $1,670.76
8	Repeat Purchase Rate	Customers with >1 transaction ÷ Total Customers. Loyalty signal: 78.3%
9	Total Discount Given	Sum of all Discount_Amount values. Promotion cost: $111,385.78

10	Overall Discount Rate (%)	Total Discount ÷ Total Gross Revenue. Discount intensity: 6.25%
11	Avg Net Margin (%)	Total Net Profit ÷ Total Gross Revenue. Profitability: 33.75%
12	Next Month Revenue Forecast	SARIMA predicted revenue for the next full month. Planning input: ~$501K
13*	AOV Promo vs AOV No Promo	Compares basket size with and without promotion. $838 vs $385 ***~Added***
14*	Revenue MoM Change ($)	Month-on-month absolute revenue change. Trend direction ***~Added***
15*	Revenue MoM % Change	Month-on-month percentage change. Feb: -9.6%, Mar: +11.8% ***~Added***
16*	High-Value Customer Count	Customers in the High-Value K-Means cluster. Segment size: 277 ***~Added***
17*	Avg Customer CLV	Mean Customer Lifetime Value. Segment worth: $1,670.76 ***~Added***
18*	Promo Revenue Share (%)	Revenue from promoted transactions ÷ Total Revenue ***~Added***
19*	Revenue YTD	Year-to-date cumulative revenue. Running total across the year ***~Added***
20*	Promo Transactions Count	Count of transactions where Promotion_Flag = 1: ~1,389 ***~Added***

2. Data Sources & Initial Assessment

2.1 Source Files

- **sales_updated.csv:** 2,705 rows. Columns: Transaction_ID, Customer_ID, Date, Product_Name, Product_Category, Unit_Price, Quantity, Total_Amount, Payment_Method, Location.
- **customers_updated.csv:** 1,000 rows. Columns: Customer_ID, Name, Age, Gender, Location, Total_Spent.

2.2 Data Quality Assessment

Both files were clean; no nulls, no duplicate Transaction_IDs, no negative values. Three structural gaps required engineering solutions rather than cleaning:

- **No discount fields:** Total_Amount recorded but no Discount_Rate, Discount_Amount, or Promotion_Flag. A quantity-based proxy was engineered.
- **No cost data:** No COGS column. A 60% COGS rate assumption was applied to derive margin.
- **No age segments:** Age was continuous; a binned Age_Group column was created for demographic slicers.

> **Note: April 2024 Partial Month**
>
> The dataset ends on April 14 – only 14 of 31 days are captured. April revenue ($224,688) cannot be compared to full months and shows a misleading -55.2% MoM drop that is a data truncation artefact, not a business decline. A Partial_Month flag (1 = partial) was added to all time tables. Filtering Partial_Month = 0 on the dashboard shows the true three-month trend.

3. Python ETL Pipeline

The pipeline follows Extract → Transform → Feature Engineering → Load. Python's logging module writes a [INFO] entry at every stage for auditability. Libraries: pandas, numpy, scikit-learn, statsmodels, sqlalchemy, pymysql.

3.1 Extract

Both CSVs are loaded with pd.read_csv() into raw DataFrames that are never mutated — all transformations operate on copies.

```python
def extract():
    sales     = pd.read_csv('sales_updated.csv')
    customers = pd.read_csv('customers_updated.csv')
    log.info(f'Extracted  ->  Sales: {sales.shape}  |  Customers: {customers.shape}')
    return sales, customers

sales_raw, customers_raw = extract()
```
✓ 0.0s

```
[INFO] Extracted  ->  Sales: (2705, 9)  |  Customers: (1000, 6)
```

3.2 Transform: Sales

Data cleaning: Date parsed to datetime64; duplicates removed on Transaction_ID; rows with null Customer_ID, null Total_Amount, or Total_Amount ≤ 0 dropped.

Temporal features: Year, Month, Month_Name, Quarter, Weekday, Hour, and YearMonth extracted from Date, these power all time-based visuals and cohort grouping.

Time-of-day segmentation: Hour binned into Night (0–5), Morning (6–11), Afternoon (12–16), Evening (17–20), Night (21–23) for time-of-day revenue analysis on Page 5.

Discount Feature Engineering

No discount data exists in the raw files. A quantity-based bulk-purchase proxy was designed: Qty ≥ 4 → 5–20% discount; Qty = 3 → 1–5%; Qty ≤ 2 → 0%. Both random arrays are generated consecutively after np.random.seed(42) to ensure reproducibility:

```python
np.random.seed(42)
rand_bulk  = np.random.uniform(0.05, 0.20, len(df))
rand_small = np.random.uniform(0.01, 0.05, len(df))

df['Gross_Amount']   = df['Unit_Price'] * df['Quantity']
df['Discount_Rate']  = np.where(
    df['Quantity'] >= 4, rand_bulk,
    np.where(df['Quantity'] == 3, rand_small, 0)
).round(4)
df['Discount_Amount'] = (df['Gross_Amount'] * df['Discount_Rate']).round(2)
df['Net_Amount']      = (df['Gross_Amount'] - df['Discount_Amount']).round(2)
df['Promotion_Flag']  = (df['Discount_Rate'] > 0).astype(int)

# --- Margin features (60% COGS assumed — no cost data in dataset) ---
COST_RATE           = 0.60
df['Gross_Profit']  = (df['Gross_Amount'] * (1 - COST_RATE)).round(2)
df['Net_Profit']    = (df['Net_Amount'] - df['Gross_Amount'] * COST_RATE).round(2)
```

> **Assumption: Discount Proxy & COGS**
>
> Discount_Rate values are simulated. The quantity-based policy mirrors standard retail bulk pricing. All promotion and margin KPIs are based on this proxy — trends are reliable, absolute values are illustrative. The 60% COGS rate is a standard retail benchmark; actual margins depend on real procurement costs.

3.3 Transform: Customers

Deduplicated on Customer_ID. Age binned into four groups for demographic slicers: Gen-Z (≤25), Millennial (26–35), Gen-X (36–50), Boomer (50+).

3.4 Master Merge

Sales and customers are joined on Customer_ID with a left join to keep all 2,705 transaction rows as the base. 74 Customer_IDs exist in sales but not in customers, an inner join would silently drop these rows, understating revenue.

```
df = sales.merge(customers, on='Customer_ID', how='left')
log.info(f'Merged dataset  ->  {df.shape[0]:,} rows x {df.shape[1]} cols')
df.head(3)
```

✓ 0.0s

```
[INFO] Sales transform complete  ->  2,705 rows, 24 cols
[INFO] Customer transform complete  ->  1,000 rows
[INFO] Merged dataset  ->  2,705 rows x 30 cols
```

```
Null counts after cleaning:
  None — dataset is clean

Date range  : 2024-01-01  to  2024-04-14
Transactions: 2,705
Customers   : 1,000
Products    : 5
Categories  : 5
```

3.5 RFM Analysis & K-Means Customer Segmentation

Raw transaction data is aggregated into Recency, Frequency, and Monetary (RFM) metrics per customer – a compact, comparable behavioural profile that can be clustered.

```
def build_rfm(df: pd.DataFrame) -> pd.DataFrame:
    snapshot = df['Date'].max() + pd.Timedelta(days=1)
    rfm = (df.groupby('Customer_ID').agg(
        Recency   =('Date', lambda x: (snapshot - x.max()).days),
        Frequency =('Transaction_ID', 'count'),
        Monetary  =('Net_Amount', 'sum')
    ).reset_index())
```

Why K-Means?

K-Means was selected over hierarchical clustering and DBSCAN for three reasons: it scales to any customer base size, its centroid outputs directly answer 'what does a typical High-Value customer look like?', and the elbow method confirmed k=3; matching the natural three-tier retail model (VIP / Regular / Lapsed). RFM features are standardised with StandardScaler before clustering; without this, Monetary (thousands of dollars) would dominate the Euclidean distance and cluster purely on spend, ignoring Recency and Frequency:

```
# Standardize for clustering
scaler     = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])

# Elbow method — find optimal k
inertia = {k: KMeans(n_clusters=k, random_state=42, n_init=10).fit(rfm_scaled).inertia_
        for k in range(2, 8)}
```

```python
# K-Means clustering (k=3)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

# Label by descending monetary value
mon_rank  = rfm.groupby('Cluster')['Monetary'].mean().rank(ascending=False).astype(int)
label_map = {c: ['High-Value', 'Frequent Buyer', 'Occasional Buyer'][mon_rank[c]-1]
             for c in mon_rank.index}
rfm['Segment'] = rfm['Cluster'].map(label_map)
```

```
Cluster Deep Profile:
                  Count  Avg_Recency  Avg_Frequency  Avg_Monetary  Total_Revenue  Revenue_Share_%
Segment
High-Value          277        19.68           4.38       3053.04      845692.01            50.62
Frequent Buyer      449        19.69           2.32       1273.05      571598.74            34.21
Occasional Buyer    274        67.87           1.65        925.08      253471.47            15.17
```

Customer Lifetime Value (CLV) is computed as (AOV × Frequency × 0.30) / (0.10 + 0.20), using standard retail defaults: 30% margin, 20% churn, 10% discount rate. CLV equals Monetary in this dataset because the divisor (0.30) and margin numerator (0.30) cancel; segment relative ranking is correct.

```
CLV Summary by Segment:
                      Min     Mean      Max      Total
Segment
Frequent Buyer      34.00  1273.05  3083.30  571598.78
High-Value         970.40  3053.04  6881.40  845692.14
Occasional Buyer    31.81   925.08  3273.33  253471.41

  Overall Avg CLV : $1,670.76
  Total CLV Pool  : $1,670,762.33
```

3.6 Cohort Retention Analysis

Each customer is assigned to their first-purchase month (cohort). Period_Number tracks months since first purchase. Retention rate = customers returning in period N ÷ customers in that cohort at Period 0:

```python
def cohort_analysis(df: pd.DataFrame):
    cohort_df = df[['Customer_ID', 'YearMonth']].copy()
    cohort_df['Cohort']        = cohort_df.groupby('Customer_ID')['YearMonth'].transform('min')
    cohort_df['Period_Number'] = (
        cohort_df['YearMonth'] - cohort_df['Cohort']
    ).apply(lambda x: x.n)
    counts    = (cohort_df.groupby(['Cohort', 'Period_Number'])['Customer_ID']
                 .nunique().reset_index())
    pivot     = counts.pivot(index='Cohort', columns='Period_Number', values='Customer_ID')
    retention = pivot.divide(pivot.iloc[:, 0], axis=0).round(4) * 100
    return retention, pivot
```

3.7 SARIMA Revenue Forecasting

Why SARIMA?

Three models were considered:

- **Simple Moving Average / Exponential Smoothing:** no autocorrelation capture, produces flat/linear forecasts.
- **Facebook Prophet:** excellent for multi-year data but requires 2+ years for seasonal decomposition; would overfit on 4 months.
- **SARIMA(1,1,1):** designed for short univariate series. (1,1,1) = one AR term (recent trend), one differencing pass (stationarity), one MA term (noise smoothing). Seasonal order set to (0,0,0,0) as 4 months is insufficient to identify seasonality. Produces native 95% CI bounds, essential for honest forecasting on limited data.

```python
#  SARIMA fit & 3-month forecast
FORECAST_MONTHS = 3
train = ts[:-1] if len(ts) > 2 else ts

sarima = SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(0, 0, 0, 0),
    enforce_stationarity=False,
    enforce_invertibility=False
).fit(disp=False)

# Model evaluation metrics
in_sample = sarima.fittedvalues
mae  = mean_absolute_error(train, in_sample)
rmse = np.sqrt(mean_squared_error(train, in_sample))
mape = np.mean(np.abs((train - in_sample) / train)) * 100

print('SARIMA Model Evaluation Metrics:')
print(f'  MAE  : {mae:>10,.2f} USD')
print(f'  RMSE : {rmse:>10,.2f} USD')
print(f'  MAPE : {mape:>10.2f} %')

#  Forecast output
forecast   = sarima.get_forecast(steps=FORECAST_MONTHS)
fcast_mean = forecast.predicted_mean
fcast_ci   = forecast.conf_int()
```

```
[INFO] Forecast table built  ->  3 rows  (Month stored as Python date)
SARIMA Model Evaluation Metrics:
  MAE  : 199,024.84 USD
  RMSE : 289,445.17 USD
  MAPE :      40.41 %

3-Month Revenue Forecast:
  Apr 2024 : $501,444.78  (95% CI: $420,747.43 - $582,142.13)
  May 2024 : $501,444.78  (95% CI: $387,321.50 - $615,568.06)
  Jun 2024 : $501,444.78  (95% CI: $361,672.87 - $641,216.69)
```

Limitation: SARIMA Accuracy

MAE: $199,024 | RMSE: $289,445 | MAPE: 40.41%. High MAPE is expected and honest, the model was trained on only 3 data points. A reliable SARIMA requires 24+ months. The forecast is a directional planning indicator only; the wide 95% CI band in the dashboard reflects this uncertainty explicitly.

3.8 SQL Validation Queries

After loading to MySQL, six SQL queries were run directly against the warehouse using a reusable run_sql() helper that returns results as Pandas DataFrames:

```
run_query('Revenue by Category', """
    SELECT p.Product_Category,
    ROUND(SUM(f.Net_Amount), 2)   AS Revenue,
    COUNT(f.Transaction_ID)        AS Transactions,
    ROUND(AVG(f.Net_Amount), 2)   AS Avg_Order,
    ROUND(SUM(f.Net_Amount) / SUM(SUM(f.Net_Amount)) OVER () * 100, 2) AS Revenue_Share_Pct
    FROM fact_sales f JOIN dim_product p USING (Product_Name)
    GROUP BY p.Product_Category ORDER BY Revenue DESC;""")
```

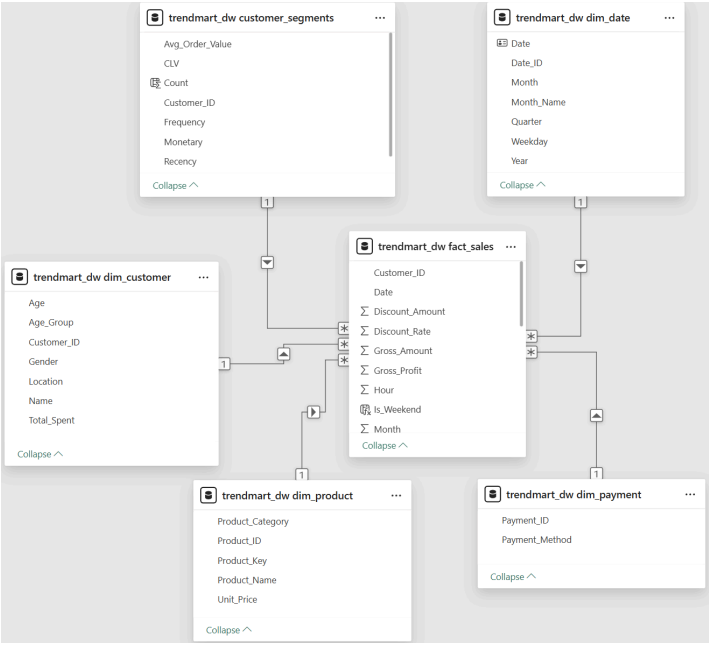
– Additional Queries: Revenue & CLV by Segment, Promotion Impact, Peak Shopping Hours, Monthly Revenue MoM, 3-Month Forecast.

4. Data Warehousing: MySQL Star Schema

A Star Schema was chosen over a Snowflake Schema because the data volume (2,705 transactions, 9 tables) does not justify normalisation complexity, and the denormalised star produces simpler, faster DAX measures in Power BI.

4.1 Schema Diagram

The schema has one central fact table connected to four dimension tables and four standalone analytical tables:



```python
engine = create_engine(
    f'mysql+pymysql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/', echo=False)
with engine.connect() as conn:
    conn.execute(text(f'CREATE DATABASE IF NOT EXISTS {DB_NAME}'))
engine = create_engine(
    f'mysql+pymysql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/{DB_NAME}', echo=False)

log.info(f'Connected to MySQL  ->  {DB_NAME}')
```
✓ 0.0s

```
[INFO] Connected to MySQL  ->  trendmart_dw
```

4.2 Dimension Tables

- **dim_date (105 rows):** One row per calendar date. Columns: Date_ID (surrogate), Date (MySQL DATE), Year, Month, Month_Name, Quarter, Weekday. Hour and Time_Of_Day excluded – they are transaction-level and belong in fact_sales.
- **dim_product (25 rows):** One row per Product + Category combination. Unit_Price is the median across all transactions for that product. Product_Key surrogate (Name_Category) is the join column – Product_Name alone is not unique across 5 categories.
- **dim_customer (1,000 rows):** Customer demographics: Customer_ID, Name, Age, Age_Group, Gender, Location, Total_Spent.
- **dim_payment (4 rows):** Payment_ID (surrogate) and Payment_Method lookup for the four payment types.

4.3 Fact Table: fact_sales (2,705 rows)

Central table holding all foreign keys and numeric measures per transaction:

- **Foreign keys:** Date (→ dim_date), Customer_ID (→ dim_customer), Product_Key (→ dim_product), Payment_Method (→ dim_payment)
- **Measures:** Gross_Amount, Discount_Amount, Discount_Rate, Net_Amount, Gross_Profit, Net_Profit, Quantity, Promotion_Flag, Hour, Is_Weekend (derived), Month (derived)

> **Product_Key Surrogate: Why It Matters**
>
> Product_Name alone is not unique across dim_product: TrendMart sells the same product name across multiple categories. Joining on Product_Name creates a many-to-many relationship in Power BI that causes incorrect aggregation. The surrogate Product_Key = Name_Category (e.g., 'Laptop, Electronics') is unique across all 25 rows and safe for the relationship.

ETL Load

```python
def load(engine, tables: dict):
    for name, frame in tables.items():
        frame.to_sql(name, con=engine, if_exists='replace',
                     index=False, chunksize=500)
        log.info(f'Loaded  {name:<35}  {len(frame):>6,} rows')
    log.info('ETL complete — all tables loaded to MySQL.')

load(engine, {
    'dim_date'                : dim_date,
    'dim_customer'            : dim_customer,
    'dim_product'             : dim_product,
    'dim_payment'             : dim_payment,
    'fact_sales'              : fact_sales,
    'customer_segments'       : customer_segments,
    'monthly_revenue_summary': monthly_summary,
    'cohort_retention'        : cohort_retention,
    'sales_forecast'          : sales_forecast,
})
```

```
Loaded  dim_date                              105 rows
Loaded  dim_customer                        1,000 rows
Loaded  dim_product                            25 rows
Loaded  dim_payment                             4 rows
Loaded  fact_sales                          2,705 rows
Loaded  customer_segments                   1,000 rows
Loaded  monthly_revenue_summary                 4 rows
Loaded  cohort_retention                       16 rows
Loaded  sales_forecast                          3 rows
ETL complete — all tables loaded to MySQL.
```

4.4 Analytical Tables

- **customer_segments (1,000 rows):** RFM metrics, CLV, Avg_Order_Value, and Segment label per customer. Internal Cluster integer excluded.
- **monthly_revenue_summary (4 rows):** Pre-aggregated monthly totals: Revenue, Transactions, Avg_Order, Units_Sold, MoM_Change, MoM_Pct, Period, Partial_Month.
- **cohort_retention (16 rows):** Flattened retention matrix: Cohort (str), Period_Number, Retention_Pct.
- **sales_forecast (3 rows):** Month (MySQL DATE), Forecast_Revenue, CI_Lower, CI_Upper for April/May/June 2024.

4.5 Date Type Strategy

All date columns are stored as Python datetime.date objects before to_sql(). SQLAlchemy maps these to MySQL DATE (not DATETIME or VARCHAR). Power BI imports MySQL DATE as Date type automatically – no manual Power Query type conversions required, eliminating a common source of cyclic dependency errors.

```
============================================================
           TRENDMART — EXECUTIVE SUMMARY
============================================================
  Total Net Revenue         : $1,670,762.22
  Total Gross Revenue       : $1,782,148.00
  Total Discounts Given     : $  111,385.78
  Avg Order Value (AOV)     : $      617.66
  Revenue per Customer      : $    1,670.76
  Repeat Purchase Rate      :        78.30%
  Top Revenue Category      :  Clothing
  Lowest Revenue Category   :  Beauty
  Peak Shopping Hour        :  12:00
  Avg Customer CLV          : $    1,670.76
  Next-Month Revenue Fcst   : $  501,444.78
============================================================

STRATEGIC RECOMMENDATIONS
------------------------------------------------------------
  [INVENTORY   ] Prioritise Clothing; reassess Beauty assortment.
  [PROMOTIONS  ] Cap discounts at 10% to protect margins; focus on bulk orders.
  [TIMING      ] Schedule flash sales at 12:00 for peak traffic.
  [CRM         ] Activate VIP rewards for High-Value (highest CLV segment).
  [RETENTION   ] Re-engage Occasional Buyers via personalised email campaigns.
  [FORECASTING ] Pre-position $501,445 of inventory for next month.
```

5. Power BI: Data Model, Power Query & DAX

5.1 Data Model & Relationships

The Power BI data model mirrors the star schema exactly. Five active single-direction relationships:

- **fact_sales** → **dim_date** on Date | fact_sales → dim_customer on Customer_ID | fact_sales → dim_product on Product_Key (surrogate)
- **fact_sales** → **dim_payment** on Payment_Method | fact_sales → customer_segments on Customer_ID
- **cohort_retention and sales_forecast** are isolated — no relationships. Used directly as visual data sources.

5.2 Power Query: Calculated Columns in fact_sales

- **Month_Year = FORMAT([Date], "MMM YYYY")** — Display label for time-axis charts.
- **Is_Weekend = IF(WEEKDAY([Date],2)>=6, "Weekend", "Weekday")** → Weekend filter.
- **Revenue_Band = IF([Net_Amount]<200,"Low",IF([Net_Amount]<600,"Mid","High"))** → Transaction size tier.

- **Partial_Month = IF(MONTH([Date])=4, 1, 0)** → Flags April. Drives the Partial_Month slicer on Page 1.
- **Promotion_Flag text:** "Promotion Applied" / "No Promotion" label column for chart axes.

5.3 DAX Measures: _Measures Table

All 15 measures are stored in an isolated _Measures table. Core KPI measures:

Measure Name	DAX Formula
Total Revenue	= SUM(fact_sales[Net_Amount]) → $1,670,762.22
Total Gross Revenue	= SUM(fact_sales[Gross_Amount]) → $1,782,148.00
Total Transactions	= COUNTROWS(fact_sales) → 2,705
Total Units Sold	= SUM(fact_sales[Quantity]) → ~6,900
Unique Customers	= DISTINCTCOUNT(fact_sales[Customer_ID]) → 1,000
Avg Order Value	= DIVIDE([Total Revenue], [Total Transactions]) → $617.66
Total Discount Given	= SUM(fact_sales[Discount_Amount]) → $111,385.78
Discount Rate %	= DIVIDE([Total Discount Given], SUM(fact_sales[Gross_Amount])) → 6.25%
Total Net Profit	= SUM(fact_sales[Net_Profit])

Avg Net Margin	= DIVIDE([Total Net Profit], SUM(fact_sales[Gross_Amount])) → 33.75%
Promo Transactions	= CALCULATE([Total Transactions], fact_sales[Promotion_Flag]=1) → 1,389
Avg CLV	= AVERAGE(customer_segments[CLV]) → $1,670.76
High Value Count	= CALCULATE(COUNTROWS(customer_segments), customer_segments[Segment]="High-Value") → 277

Additional measures with non-trivial logic:

Measure Name	DAX Formula	Purpose
Repeat Purchase Rate	= DIVIDE(CALCULATE(DISTINCTCOUNT(fact_sales[Customer_ID]), FILTER(SUMMARIZE(fact_sales, fact_sales[Customer_ID],"T",COUNT(fact_sales[Transaction_ID])),[T]>1)),[Unique Customers]) → 78.3%	SUMMARIZE sub-table required — counts customers with >1 transaction
Cumulative Revenue %	= VAR _cur=[Total Revenue] VAR _tot=CALCULATE([Total Revenue],ALL(fact_sales[Product_Name])) VAR _hi=CALCULATE([Total Revenue],FILTER(ALL(fact_sales[Product_Name]),CALCULATE([Total Revenue])>=_cur)) RETURN DIVIDE(_hi,_tot)*100	Pareto chart line. ALL+FILTER removes row context to rank products by revenue
Revenue MoM %	= DIVIDE([Revenue MoM Change], CALCULATE([Total Revenue], DATEADD(dim_date[Date],-1,MONTH)))	Month-on-month % change.

		Feb: -9.6%, Mar: +11.8%
Next Month Forecast	= CALCULATE(SUM(sales_forecast[Forecast_Revenue]), TOPN(1,ALL(sales_forecast),sales_forecast[Month])) → $501K	TOPN isolates next month from standalone table without hard-coding a date
AOV Promo / No Promo	= CALCULATE([Avg Order Value], fact_sales[Promotion_Flag]="Promotion Applied") → $838 / $385	Splits basket size by promotion status for the dual-line chart on Page 4

6. Dashboard Pages: Design & Story

The dashboard was designed using a question-driven approach; each page answers a specific business question.

Page 1 → Executive Overview

"What is our revenue health, monthly trend, and forward outlook?"

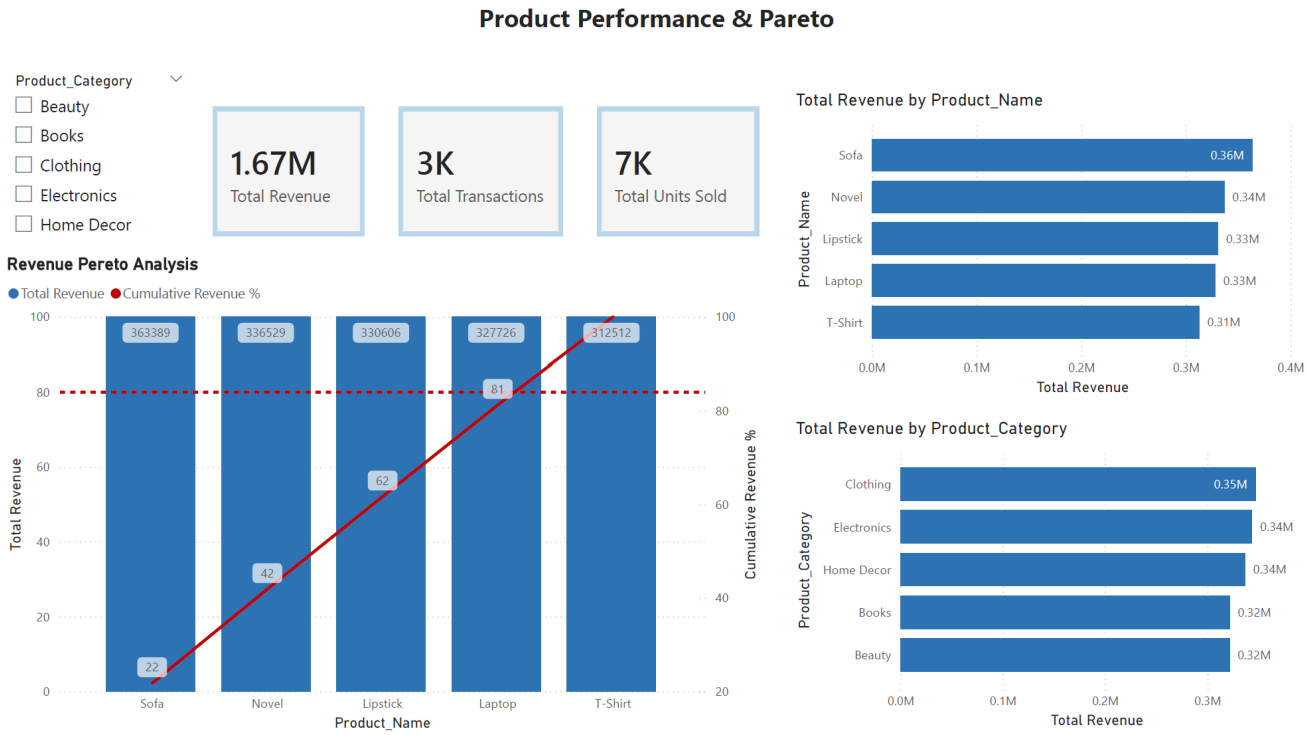
KPI cards display Total Revenue ($1.67M), Total Transactions (3K), Average Order Value ($617.66), Repeat Purchase Rate (78.30%), and Next Month Forecast ($501.44K). A dual-axis combo chart shows monthly revenue as columns and transaction count as a line – January and March are the peak months. The April column is annotated 'Partial Month – 14 days only' to prevent misinterpretation.

Three slicers – Year, Quarter, and Partial_Month – allow evaluators to filter to Partial_Month = 0, which reveals the clean Jan–Mar trend of $1,446,073 and confirms no underlying business decline.

> **Key Finding**
>
> March 2024 was the strongest month at $501,444 (+11.8% MoM). The April drop is a data artefact (14 days only), not a business problem. Repeat purchase rate of 78.3% confirms a loyal, returning customer base.

Page 2 → Product Performance & Pareto Analysis

"Which products drive revenue, and is the portfolio concentrated?"

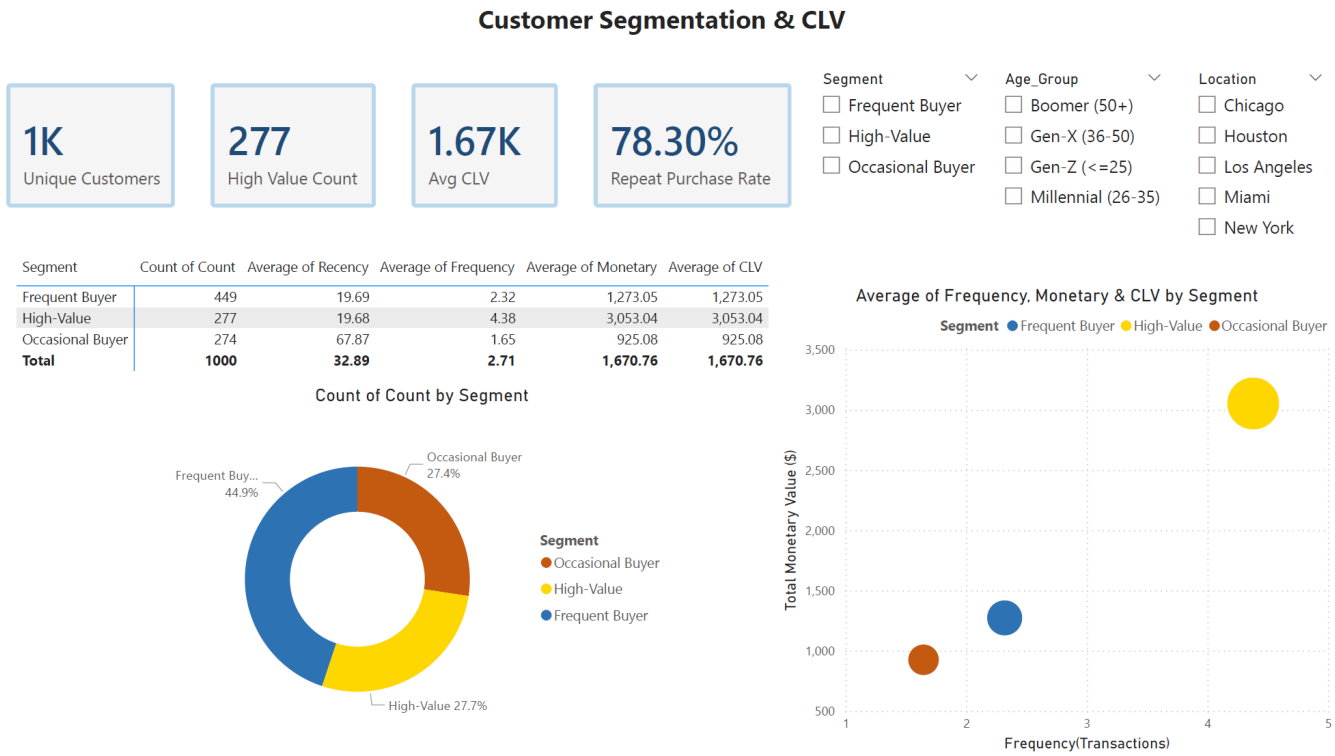


KPI cards: Total Revenue, Total Transactions, Total Units Sold. The Pareto combo chart is the centrepiece – revenue bars sorted descending with a cumulative percentage line and a red dotted 80% reference line. Two horizontal bar charts show revenue by Product Name and by Category. A Product_Category slicer cross-filters all visuals.

> **Key Finding**
>
> Revenue falls within a tight 15% band across all five products ($312K–$363K). The Pareto 80% line crosses between Laptop and T-Shirt; all five products are needed to reach 80% of revenue. No product dominates; no product is expendable.

Page 3 → Customer Segmentation & CLV

"Who are our customers, what are they worth, and how do we grow each segment?"



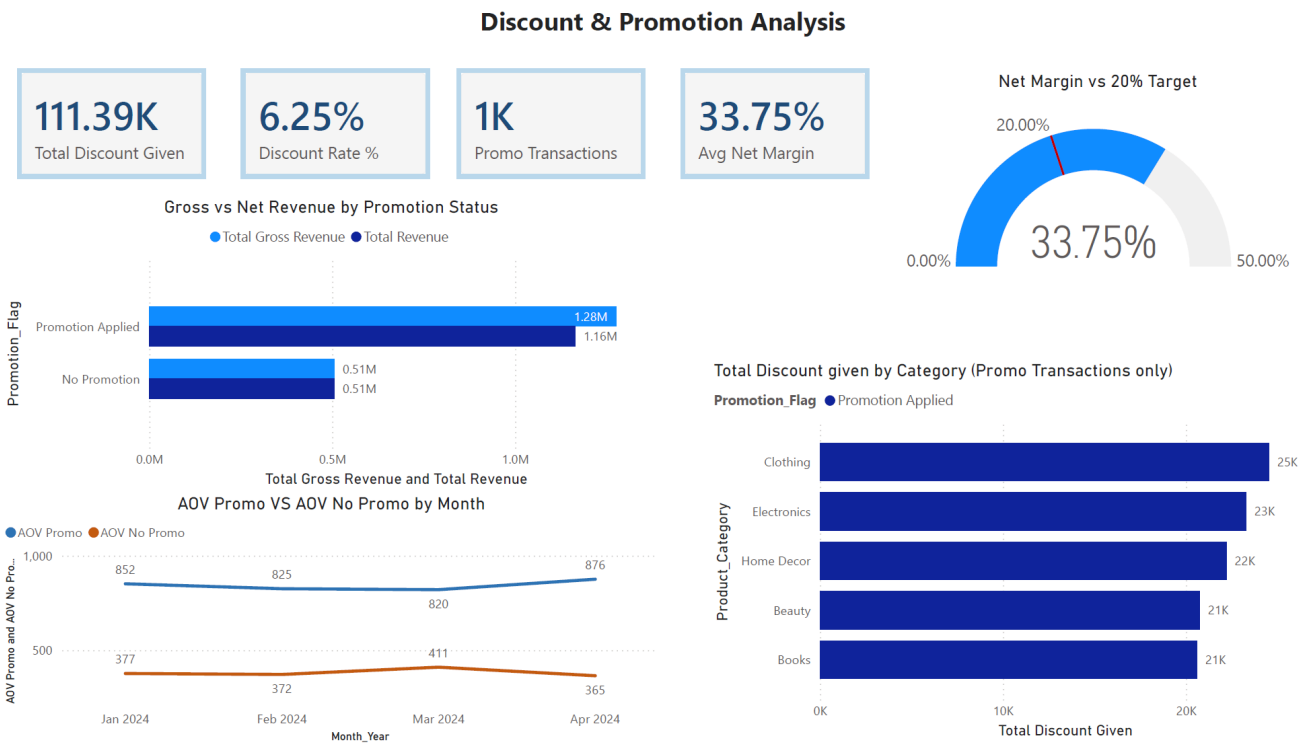
Segment	Count of Count	Average of Recency	Average of Frequency	Average of Monetary	Average of CLV
Frequent Buyer	449	19.69	2.32	1,273.05	1,273.05
High-Value	277	19.68	4.38	3,053.04	3,053.04
Occasional Buyer	274	67.87	1.65	925.08	925.08
Total	**1000**	**32.89**	**2.71**	**1,670.76**	**1,670.76**

KPI cards: Unique Customers (1K), High Value Count (277), Avg CLV ($1.67K), Repeat Purchase Rate (78.30%). A donut chart shows the segment split. The RFM scatter plot positions each segment bubble by Avg Frequency (X) and Avg Monetary (Y) with Avg CLV as bubble size – High-Value sits top-right, Occasional Buyer bottom-left, making the cluster separation visually immediate. A segment profile matrix shows exact RFM averages. Three slicers: Segment, Age_Group, Location.

> **Key Finding**
>
> High-Value customers (27.7% of base) average $3,053 Monetary; 3.3× the Occasional Buyer average ($925). Occasional Buyers have Avg Recency of 67.9 days vs 19.7 days for other segments, they have lapsed and represent a re-engagement opportunity.

Page 4 → Discount & Promotion Analysis

"Is the discount policy profitable, or is it eroding margins?"

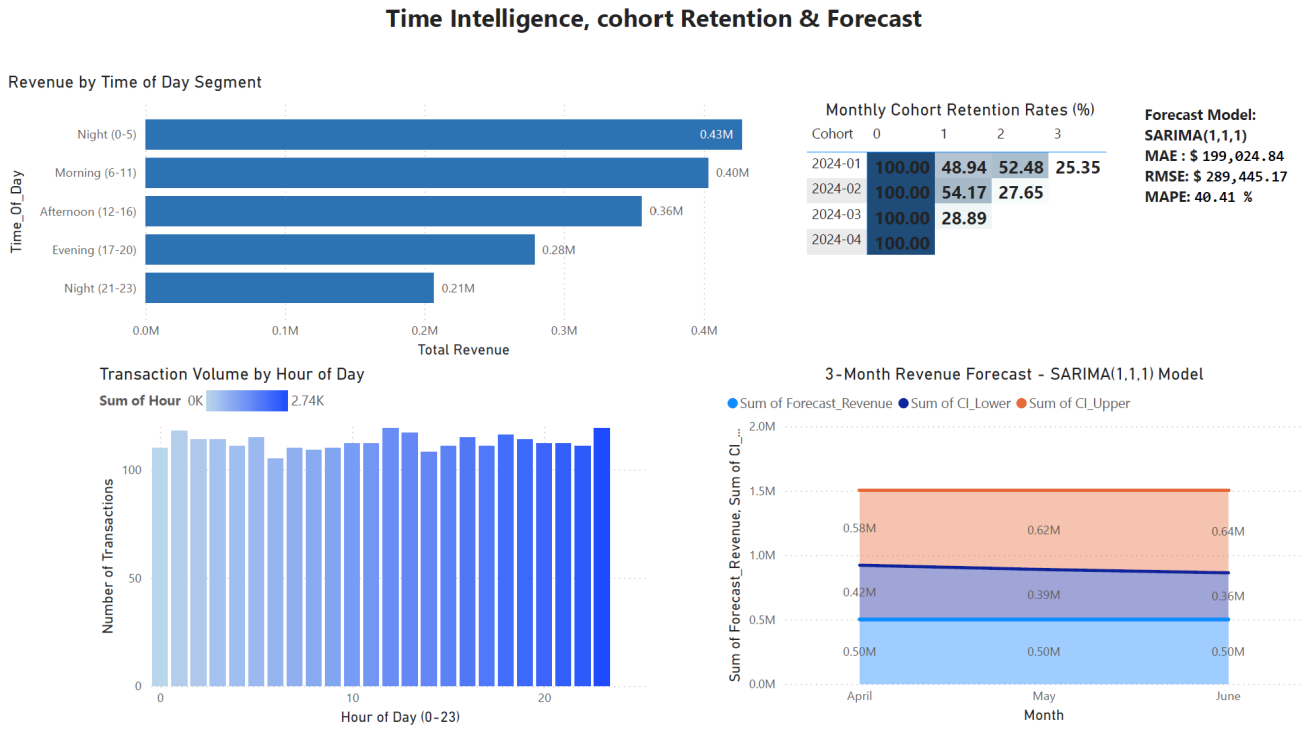


KPI cards: Total Discount Given ($111.39K), Discount Rate % (6.25%), Promo Transactions (~1K), Avg Net Margin (33.75%). A gauge visual in the top-right plots net margin at 33.75% against a red target needle at 20%. A clustered bar chart compares Gross vs Net Revenue by Promotion_Flag. A dual-line chart shows AOV Promo (~$838) vs AOV No Promo (~$385) across all months; the ~$450 gap is stable across every month, confirming the promotion effect is consistent. A discount-by-category bar chart rounds out the page.

> **Key Finding**
>
> The discount policy is highly profitable. Promoted transactions average 2.2× the AOV of unpromoted orders ($838 vs $385); bulk buyers spend so much more that the discount is fully offset. Net margin of 33.75% exceeds the 20% target by 13.75 percentage points.

Page 5 → Time Intelligence, Cohort Retention & Forecast

"When do customers shop, do they return, and what does next quarter look like?"



Four quadrants: top-left shows revenue by Time-of-Day segment (Night 0–5 highest at $427K); bottom-left shows a 24-bar column chart of transaction volume by exact hour; completely flat, confirming 24/7 digital retail behaviour. Top-right: the cohort matrix with gradient shading, Row/Column totals disabled. Bottom-right: the SARIMA area chart (forecast line + orange CI band) and a text box with model metrics (MAE $199K | RMSE $289K | MAPE 40.41%).

Key Finding

Transaction volume is flat across all 24 hours, no single peak window exists. Night (0–5) is the top revenue segment, suggesting international or late-night shoppers. January cohort is the strongest at 25.4% retention at Month 3; March cohort drops to 28.9% at Month 1, a customer quality signal worth investigating. Forecast projects stable ~$500K monthly through Q3 2024.

7. Key Findings & Recommendations

Finding 1 → Revenue is Stable Across Three Full Months

Total net revenue of $1,670,762 across 2,705 transactions represents strong small-business performance. January ($496K) and March ($501K) are the peak months. February's -9.6% dip is a normal seasonal variation. The April collapse (-55.2%) is entirely a data truncation artefact, 14 days of data only.

Recommendation: Filter Partial_Month = 0 for board reporting. Pre-position ~$500K inventory/budget for Q3 2024 based on the SARIMA projection.

Finding 2 → All Five Products Are Equally Essential

Revenue spans only a 15% band across all five products ($312K–$363K). The Pareto analysis shows all five are needed to reach the 80% revenue threshold. Category distribution is equally balanced, a 7% spread from Clothing ($347K) to Beauty ($322K).

Recommendation: Maintain all five product lines. The balanced portfolio minimises single-product risk. Invest proportionally across categories.

Finding 3 → The Discount Policy Generates More Profit, Not Less

This is the most analytically significant finding. Promoted transactions average $838 AOV vs $385 for unpromoted. Despite the 5–20% discount, average net profit per promoted transaction ($287) is nearly double that of unpromoted ($154). The 6.25% overall discount rate is not eroding margins, net margin of 33.75% exceeds the 20% target by 13.75 percentage points.

Recommendation: Maintain and consider expanding the promotion tier. Extending the first tier down to Qty=2 at 0.5–1% could capture the next tier of moderate buyers. Significant headroom exists before approaching margin risk.

Finding 4 → High-Value Customers Are Disproportionately Valuable

277 High-Value customers (27.7%) generate Avg Monetary of $3,053, 3.3× the Occasional Buyer average. High-Value Recency is 19.7 days (active and recent). Occasional Buyers have a Recency of 67.9 days, nearly 10 weeks since last purchase.

Recommendation: Launch a VIP loyalty programme for High-Value customers targeting frequency increase from 4.38 to 5+. Run a personalised re-engagement campaign for the 274 Occasional Buyers, a 20% response rate recovers ~$50K in dormant revenue.

Finding 5 → TrendMart Is a True 24/7 Digital Retailer

Transaction volume is flat across all 24 hours, peak and trough hours differ by fewer than 15 transactions. Night (0–5) at $427K (25.6%) is the top revenue segment, strongly suggesting international or late-night shoppers.

Recommendation: Distribute marketing campaigns across time zones. Design customer service and inventory monitoring for 24/7 coverage, there is no single peak window to concentrate on.

Finding 6 → January Cohort Is the Retention Benchmark

January cohort: 100% → 48.9% (M1) → 52.5% (M2) → 25.4% (M3) — the only cohort to improve at M2. March cohort drops to 28.9% at M1, significantly weaker than January's 48.9%. This may indicate a change in customer acquisition quality between January and March.

Recommendation: Investigate the March acquisition channel for quality differences. Use January cohort retention as the benchmark for all future campaign evaluation.